

Übung 4: Advanced Java, Test-Driven Design (TDD) und LLM-gestütztes Entwickeln

Team 6 (Thema E-Vote) - Abgabe:

Link: https://github.com/jacque-schu/MSE_eVote

Gemeinsam als Team haben wir uns über Discord zum Thema „Übung 4: Advanced Java, Test-Driven Design (TDD) und LLM-gestütztes Entwickeln“ ausgetauscht (in unserem Fall jedoch Python statt Java). Für die folgenden Aufgaben haben wir in einem Google-Docs Dokument unsere Antworten zusammengetragen.

Aufgabe 1: Review Ihrer Implementierungsstrategie:

Diskutieren Sie im Team (a) die grundlegenden Konzepte von Continuous Integration und Continuous Deployment und (b) wie CI/CD Ihre Softwareentwicklung unterstützt und wie automatisierte Tests und Builds in den Workflow integriert werden können. Halten Sie Ihre Diskussionsresultate fest, um diese in der Zwischenpräsentation zu vorzustellen.

Diese Diskussion ist noch fortlaufend, wir halten verschiedene Ideen/Konzepte im Miro-Board fest. Zudem kommt als Nachtrag dazu, dass wir aus dem letzten Übungsblatt 3 noch die letzte Teilaufgabe „6. Implementierungsstrategie“ offen hatten, die wir nun angehen konnten. Da wir in unserem Project mit Python arbeiten, hatten wir für die praktische Umsetzung verschiedene Frameworks zur Auswahl. Wir entschieden uns letztendlich für FastAPI, welches nun das Gerüst für unsere Implementierung bildet.

Aufgabe 2: Testfälle mit LLM generieren und validieren (TDD Schritt 1):

Der erste zentrale Teil unserer Domänenlogik ist die Registrierung eines neuen Bürgers (BC: Bürgerverwaltung). Die Tests wurden mittels LLM-gesteuerter Generierung von Testfällen erstellt, mit Fokus auf Validierungsmethoden, insbesondere zur: Eingabevalidierung von Name, Adresse, Geburtsdatum und Email.

Die Tests decken folgende Kategorien ab:

- Happy Path: Typische zulässige Eingaben

- Edge Cases: Grenzfälle, z.B. sehr kurze oder lange Namen, Sonderfälle bei Datum, etc.
- Negative Tests: Ungültige Eingaben wie Zahlen im Namen, spezielle Sonderzeichen, falsche Formate

Für die Namensvalidierung wurde ergänzend eine Regex eingeführt, die nur Buchstaben (auch Unicode), Leerzeichen und Bindestriche erlaubt – dabei wird auch geprüft, dass Namen nicht mit Bindestrich beginnen oder enden.

Die Geburtsdatumseingabe wurde an das Format „TT.MM.JJ“ angepasst und per Eigen-Parser im Validator behandelt, sodass Strings korrekt in Python date konvertiert und validiert werden.

Auch für den zweiten (Abstimmungsübersicht) und dritten (Ergebnisdienst) Teil unserer Domänenlogik wurden Testfälle generiert, die die oben genannten Kategorien abdecken. Was genau wird zum Beispiel in Teil 3 (Ergebnisdienst) getestet?

Happy Path:

- Gültige Stimmenanzahlen (z.B. anzahl=5, anzahl=0, anzahl=100000) werden akzeptiert und korrekt zugewiesen
- Gültige Stimmoptionen ("Ja", "Nein", "Enthaltung") funktionieren ohne Fehler
- Das "Ergebnis" entsteht bei vorhandenen Einzelwerten und die Methoden liefern die korrekten Resultate

Edge Cases:

- Grenzwerte wie null Stimmen (anzahl=0) oder extrem hohe Werte (anzahl=100000), sowie Minimalbelegung für Einzelwerte (Liste mit genau einer Stimme) werden explizit getestet
- Case Sensitivity (z.B. "ja" statt "Ja") wird als Grenzfall für Enums geprüft

Negative Tests:

- Ungültige Werte wie negative Zahlen bei Stimmen, unzulässige oder falsch geschriebene Stimmoptionen, leere oder None-Einzelwerte führen zu ValidationError/Fehlern und werden bewusst erwartet. Es gibt zahlreiche weitere Testfälle, die das LLM empfiehlt und je nach Bedarf im Laufe des Projektes ergänzt werden können. Auf ein Beispiel wird in 4. Eingegangen.

Aufgabe 3: Implementierung der Domänenlogik (TDD Schritt 2) mit LLM-Pair-Programming:

Zu allen drei Teilen der Dämonenlogik wurde Code mit Hilfe eines LLMs (perplexity pro) generiert, der die unter 2. erstellten Tests besteht. Das LLM schlägt einige Verbesserungen vor. Zum Beispiel gilt es Redundanzen zu vermeiden.

Beispiel aus ergebnis.py:

```
def gesamtergebnis(self) -> int: return sum(e.anzahl for e in self.einzelwerte)

def getGesamtergebnis(self) -> int: return self.gesamtergebnis
```

Hier empfiehlt das LLM die Methode getGesamtergebnis() zu entfernen und konsistent das gesamtergebnis (Property) zu nutzen. Demzufolge wurde sowohl ergebnis.py als auch test_validation_ergebnis.py entsprechend angepasst. Auf Nachfrage an das LLM ob dann die Methode getErgebnisDetails() ebenfalls als property verwendet werden sollte, bestätigte dies das LLM, falls keine Parameter nötig wären. Da noch nicht genau feststeht, welche Ausgaben letztendlich benötigt werden, wird getErgebnisDetails() vorerst so gelassen.

Eine weitere beispielhafte Empfehlung des LLMs ist die Nutzung von Enum-Strings bei Ausgaben anstelle von Enum-Objekten. Dies ist sinnvoll für die Ausgabe und Serialisierung z.B. beim Erstellen von Dicts, bei der Anzeige für Nutzer oder beim Export/Import. Deshalb wurde folgender Code angepasst.

Vorher:

```
def getErgebnisDetails(self) -> List[dict]: return [ {"Option": e.stimmoption.optiontext,
"Stimmen": e.anzahl} for e in self.einzelwerte ]
```

Nachher:

```
def getErgebnisDetails(self) -> List[dict]: return [ {"Option":
e.stimmoption.optiontext.value, "Stimmen": e.anzahl} for e in self.einzelwerte ]
```

Das LLM macht durchaus sinnvolle Vorschläge zum Verbessern des Codes. Allerdings muss immer geprüft werden, ob diese entsprechend benötigt werden, da das LLM nicht das gesamte Projekt im Auge hat und ggf. nicht über alle Informationen verfügt.

Aufgabe 4: Tests-Erweiterung und Refaktorisierung (TDD Schritt 3):

Zu allen drei Teilen der Dämonenlogik wurde das LLM nach Testerweiterungen gefragt. Das LLM schlägt zum Beispiel vor, extrem hohe Wertebereiche zu testen.

Beispiel aus ergebnis.py: Großer Wertebereich für "anzahl"

- Extrem hohe Zahlen könnten zu Performance- oder Logikfehlern führen (z.B. bei Integer Overflow oder Datenbankgrenzen)
- Test: Erzeuge "Stimmenanzahl" mit sehr großem "anzahl" (z.B. 10^{12})

```
def test_stimmenanzahl_large_value():
```

```
    option = Stimmoption(optiontext=Optionen.JA)
```

```
    großer_wert = 10**12 # Beispiel: eine Billion Stimmen
```

```

stimme = Stimmenanzahl(stimmoption=option, anzahl=großer_wert)

assert stimme.anzahl == großer_wert

stimmen = [Stimmenanzahl(stimmoption=option, anzahl=großer_wert)]

ergebnis = Ergebnis(ergebnisID=1, abstimmungsID=99, einzelwerte=stimmen,
timestamp=datetime.now())

assert ergebnis.gesamtergebnis == großer_wert

```

Weitere Testfälle werden sich im Laufe des Projektes ergeben. Einige Vorschläge des LLMs wurden nicht umgesetzt, da noch nicht ganz klar ist, welche Ausgaben letztendlich benötigt werden. Diese Vorschläge werden dann erneut beurteilt.

Aufgabe 5: Modularität und Testbarkeit via CI/CD sicherstellen:

Unsere CI/CD Pipeline hat nach Implementierung der Inhalte dieses Aufgabenblattes Fehler angezeigt. Daher wurden folgende Änderungen vorgenommen:

- CI-Definition: Ein neuer Workflow unter `.github/workflows/ci.yml` führt bei push und pull_request automatisch Build und pytest aus; fehlende Abhängigkeiten wurden in requirements ergänzt.
- Pfade/Imports: Paketstruktur und `init.py`-Dateien wurden ergänzt, damit die Tests die Domänenmodule zuverlässig finden.
- Pydantic-Listen: Mutable-Default-Falle behoben durch `Field(default_factory=list)` für teilnehmerliste und stimmen.
- Testabdeckung: Edge-Cases ergänzt (keine Stimmen, doppelte Stimmen, Stimmen außerhalb Zeitraum, Statuswechsel, identische Start-/Enddaten, ungültige Optionen).

Aufgabe 6: Kritische Reflektion zu TDD, DDD und LLM-gestützte Entwicklung:

- Testgetriebene Entwicklung (TDD) führt zu einem strukturierten und fehlerärmeren Entwicklungsprozess, da Anforderungen frühzeitig überprüft werden. Sie verbessert die Codequalität und erleichtert Refactoring, erfordert jedoch anfangs mehr Aufwand und stößt bei komplexen Systemen an Grenzen, wenn geeignete Tests schwer vorab zu definieren sind.
- Domain-Driven Design (DDD) unterstützt das Team dabei, eine gemeinsame Sprache und klare Struktur im Projekt zu etablieren. Durch die Aufteilung in klar abgegrenzte Domänen wissen alle Beteiligten jederzeit, in welchem Bereich sie arbeiten und welche Logik betroffen ist. Dies erleichtert Abstimmung, Testabdeckung und Wartbarkeit. Gleichzeitig erfordert DDD eine konsequente Einhaltung der definierten Domänengrenzen, um die Übersicht zu bewahren.

- Large Language Models (LLMs) beschleunigen die Entwicklung, etwa durch Codegenerierung oder Ideenvorschläge, bergen jedoch das Risiko unkritischer Übernahme. Eine reflektierte Nutzung ist daher essenziell, um die generierten Vorschläge sinnvoll in den eigenen Kontext zu integrieren. LLMs eignen sich besonders für standardisierte Aufgaben, weniger für komplexe, domänenspezifische Logik.

Fazit: TDD, DDD und LLMs ergänzen sich, wenn ihre jeweiligen Stärken gezielt genutzt und ihre Grenzen beachtet werden. Ein bewusster Umgang mit LLMs trägt dazu bei, Codequalität, Verständlichkeit und Teamkonsistenz nachhaltig zu sichern.

All unsere Lösungen sind ebenfalls auf GitHub in unserer ReadMe-Datei festgehalten, siehe Link: https://github.com/jacque-schu/MSE_eVote